

Q.Types of Constraints That Can Be Applied on Table

1. Column Level Constraints

- **NOT NULL**
- **UNIQUE**
- **PRIMARY KEY**
- **CHECK**
- **DEFAULT**
- **FOREIGN KEY**

2. Table Level Constraints

- Composite PRIMARY KEY
- Composite UNIQUE
- FOREIGN KEY
- CHECK conditions involving multiple columns

```
CREATE TABLE Student (  
  id INT PRIMARY KEY,          -- 1 constraint  
  email VARCHAR(50) UNIQUE,    -- 2  
  name VARCHAR(50) NOT NULL,   -- 3  
  age INT CHECK (age >= 18),   -- 4  
  course_id INT,  
  CONSTRAINT fk_course FOREIGN KEY(course_id) REFERENCES Course(id) --5  
);
```

Q Main Difference unique & primary key

- Primary Key uniquely identifies each record and does NOT allow NULL.
- Unique Key also ensures uniqueness but allows ONE NULL value (in most DBs).

Q Difference Between ViewData, ViewBag, TempData, and Session

Feature	ViewData	ViewBag	TempData	Session
Type	Dictionary	Dynamic Object	Dictionary	Key-Value
Namespace	Microsoft.AspNetCore.Mvc	Microsoft.AspNetCore.Mvc	Microsoft.AspNetCore.Mvc	Microsoft.AspNetCore.Http
Scope	Controller → View	Controller → View	Controller → Redirect → View	Across entire session
Lifetime	Current Request	Current Request	One Request (After Redirect)	Until Session Ends
Type Casting	Required	Not Required	Required	Required
Data Storage	ViewDataDictionary	ViewDataDictionary	Session/Cookie	Server Memory
Usage	Small data to View	Simple data to View	Messages after redirect	User-specific data

Q What is ViewData, ViewBag, TempData, Session

1. ViewData

- **ViewData** is a dictionary object used to pass data from Controller to View.
ViewData["Message"] = "Welcome to ASP.NET Core";
In View:
@ViewData["Message"]

Characteristics

- Type: **ViewDataDictionary**
- Uses key-value pairs
- Requires type casting
- Works only for current request

Where We Use ViewData

- Passing simple data from Controller to View
- Displaying page title
- Showing small messages

Example:

ViewData["Title"] = "Student List";

2. ViewBag

ViewBag is a dynamic wrapper around ViewData used to pass data from Controller to View.

```
ViewBag.Message = "Welcome User";
```

In View:

```
@ViewBag.Message
```

Characteristics

- Uses dynamic properties
- No type casting required
- Works only for current request

Where We Use ViewBag

- Passing simple data
- UI messages
- Dropdown lists

Example:

```
ViewBag.City = "Surat";
```

3. TempData

TempData is used to pass data between two requests, usually after redirect.

```
TempData["Success"] = "Data saved successfully";
```

In View:

```
@TempData["Success"]
```

Characteristics

- Type: **ViewDataDictionary**
- Uses key-value pairs
- Requires type casting
- Stores data temporarily
- Uses Session internally
- Data removed after one request

Where We Use TempData

- Success messages after redirect
- Form submission confirmation

Example:

```
TempData["Message"] = "Record inserted successfully";  
return RedirectToAction("Index");
```

4. Session

Session is used to store **user-specific data across multiple requests**.

```
HttpContext.Session.SetString("UserName", "Kailas");
```

Retrieve:

```
string name = HttpContext.Session.GetString("UserName");
```

Characteristics

- Stored on **server**
- Available **throughout user session**
- Requires **session configuration**

Where We Use Session

- User login data
- Shopping cart
- User preferences

Example:

1. Login – Fetch Credentials from PostgreSQL and Create Session

```
using Microsoft.AspNetCore.Mvc;  
using Npgsql;
```

```
public class AccountController : Controller  
{
```

```
    string connString =  
    "Host=localhost;Port=5432;Username=postgres;Password=1234;Database=db_test";
```

```
    [HttpPost]
```

```
    public IActionResult Login(string username, string password)
```

```
    {  
        using (NpgsqlConnection con = new NpgsqlConnection(connString))  
        {  
            con.Open();
```

```
            string query = "SELECT id, username FROM users WHERE username=@username AND  
password=@password";
```

```

using (NpgsqlCommand cmd = new NpgsqlCommand(query, con))
{
    cmd.Parameters.AddWithValue("@username", username);
    cmd.Parameters.AddWithValue("@password", password);

    NpgsqlDataReader reader = cmd.ExecuteReader();

    if (reader.Read())
    {
        // Store session after successful login
        HttpContext.Session.SetString("UserId", reader["id"].ToString());
        HttpContext.Session.SetString("UserName", reader["username"].ToString());

        return RedirectToAction("Dashboard");
    }
    else
    {
        ViewBag.Message = "Invalid Username or Password";
        return View();
    }
}
}
}
}

```

2. Dashboard – Check Session

If session does not exist → redirect to Login.

```

public IActionResult Dashboard()
{
    string userId = HttpContext.Session.GetString("UserId");

    if (userId == null)
    {
        return RedirectToAction("Login");
    }

    return View();
}

```

3. Logout – Clear Session

```

public IActionResult Logout()
{

```

```
HttpContext.Session.Clear();  
return RedirectToAction("Login");  
}
```

Q Properties of a Class in C#

Syntax of Property

```
class Student  
{  
    private string name;    // field  
  
    public string Name      // property  
    {  
        get { return name; }  
        set { name = value; }  
    }  
}
```

Access

```
Student s = new Student();  
s.Name = "Rahul";        // set  
Console.WriteLine(s.Name); // get
```

Q What are Service Lifetimes in ASP.NET Core

Type	Object Created
AddSingleton	One object for whole application
AddScoped	One object per request
AddTransient	New object every time

1. What is **AddSingleton** in ASP.NET Core?

`builder.Services.AddSingleton<BookHelper>();`

AddSingleton registers a service with the Singleton lifetime, meaning only one instance of the service is created for the entire application lifetime.

`builder.Services.AddSingleton<BookHelper>();`

Behaviour

- The instance is created only once when the application starts.
- The same instance is used for all users and requests.
- It remains alive until the application stops.

Why is **AddSingleton** dangerous for Database Connections?

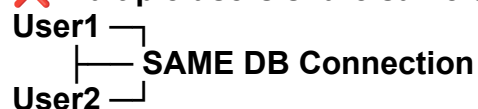
Using **AddSingleton** with database connections like **NpgsqlConnection** is dangerous.

Example (Wrong):

`builder.Services.AddSingleton<NpgsqlConnection>();`

Problems

✗ Multiple users share same connection



✗ Connection conflicts

- One request may close the connection while another is using it.

✗ Thread safety issues

- Database connections are not thread-safe.

✗ Data corruption risk

Real-world Example

Correct DI configuration:

`builder.Services.AddScoped<BookHelper>();`

Or

```
builder.Services.AddScoped<ICrudRepository, CrudRepository>();
```

Each request gets a new instance, avoiding concurrency issues.

B) AddScoped (BEST FOR DB)

```
builder.Services.AddScoped<BookHelper>();
```

Behaviour

- New instance per HTTP request
- Same object used inside that request
- Next request → new object

✓ Best for:

- DbContext
- NpgsqlConnection
- Repository
- Session based work

Question: Where do we use Singleton and Scoped?

Answer:

- Singleton is used for services that should have only one instance for the entire application, such as caching, logging, and configuration services.
- Scoped is used for services that should be created once per HTTP request, such as repositories, business services, and database connections.

C) AddTransient

```
builder.Services.AddTransient<BookHelper>();
```

Behaviour

- Every time injected → new object

Example:

```
public class TestController
{
    public TestController(BookHelper a, BookHelper b)
```

👉 a and b are DIFFERENT objects

✓ Good for:

- Light weight services
- Email sender
- Helper utilities

Difference Between Scoped and Transient

Feature	Scoped	Transient
Instance Creation	One per request	New every time
Lifetime	Per HTTP request	Per usage
Performance	Better than transient	More object creation
Sharing in Request	Same instance in same request	Different instance each time
Common Use	Repository, DB operations	Lightweight services

4 Example With Your Project

Scenario

Controller calls BookHelper

```
public MVCBookController(BookHelper bookHelper)
{
    _bookHelper = bookHelper;
}
```

If Singleton ❌

All users share:

- same _conn
- same state

- crash possible
-

If Scoped  (CORRECT)

Each request:

- own BookHelper
 - own NpgsqlConnection
 - safe
-

If Transient 

Every method call:

- new BookHelper
 - more memory
 - unnecessary objects
-

Side-by-Side Example

// 1. Singleton
AddSingleton()

User1 → same object
User2 → same object
User3 → same object

// 2. Scoped
AddScoped()

User1 → Obj1
User2 → Obj2
User3 → Obj3

// 3. Transient
AddTransient()

Call1 → Obj1
Call2 → Obj2
Call3 → Obj3

6) WHAT TO USE WHEN

Scenario	Use
Database connection	Scoped
DbContext	Scoped
Logging	Singleton
Cache	Singleton
Email service	Transient
Utility class	Transient

7) Your Correct Configuration

```
builder.Services.AddScoped<NpgsqlConnection>();  
builder.Services.AddScoped<BookHelper>();
```

8) Interview Perfect Answer

AddSingleton – one instance for entire app

AddScoped – one instance per request

AddTransient – new instance every time

For database operations Scoped is best to avoid connection conflicts.

Q What is ExecuteScalar, ExecuteNonQuery, ExecuteReader

✓ 1. ExecuteScalar()

👉 Used when query returns single value (first row, first column).

- Returns: `object`
- Best for: COUNT, SUM, MAX, MIN, single value queries.

Example

```
SqlCommand cmd = new SqlCommand("SELECT COUNT(*) FROM Student", con);  
int total = (int)cmd.ExecuteScalar();
```

Get Student Name by ID

```
SqlCommand cmd = new SqlCommand(  
    "SELECT name FROM Student WHERE id = 1", con);
```

```
string name = cmd.ExecuteScalar().ToString();
```

```
Console.WriteLine("Name = " + name);
```

✓ Fastest when only one value is needed.

✓ 2. ExecuteNonQuery()

👉 Used for queries that do NOT return data.

- Used with: INSERT, UPDATE, DELETE, CREATE, ALTER
- Returns: **number of rows affected (int)**

Example

```
SqlCommand cmd = new SqlCommand(  
    "UPDATE Student SET name='Rahul' WHERE id=1", con);
```

```
int rows = cmd.ExecuteNonQuery();
```

```
Console.WriteLine(rows + " record updated");
```

✓ Used for data manipulation.

✓ 3. ExecuteReader()

👉 Used when query returns multiple rows and columns.

- Returns: `SqlDataReader`
- Forward-only, read-only data stream.

Example

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Student", con);
```

```
SqlDataReader dr = cmd.ExecuteReader();
```

```
while (dr.Read())
{
    Console.WriteLine(dr["name"]);
}
dr.Close();
```

✓ Best for displaying list / table data.

Q What is ADO.NET?

ADO.NET (ActiveX Data Objects for .NET) is a data access technology in the .NET Framework used to **connect, retrieve, insert, update and delete data** from databases like SQL Server, Oracle, PostgreSQL, MySQL, etc.

👉 It acts as a **bridge between C# application and database**.

Main Goals of ADO.NET

- Connect to database
 - Execute SQL commands
 - Fetch data
 - Work in connected & disconnected mode
 - Provide high performance and security
-

Q Types of ADO.NET Architecture

ADO.NET works in **two models**:

1. Connected Architecture(Real-time connection)

Connection remains OPEN while reading data.

Classes Used

- Connection (SqlConnection / NpgsqlConnection)
- Command (SqlCommand/NpgsqlCommand)
- DataReader
- DataAdapter (partially)

Features

- ✓ Fast performance
- ✓ Real-time data
- ✓ Low memory usage
- ✗ Connection must remain open

Example Objects

- ExecuteReader()
- ExecuteScalar()
- ExecuteNonQuery()

PostgreSQL Example (Connected)

✓ 1. Connection + ExecuteScalar

```
using Npgsql;

string connString =

"Host=localhost;Port=5432;Username=postgres;Password=1234;Database=db_test";

// ---- CONNECTION ----
NpgsqlConnection con = new NpgsqlConnection(connString);

con.Open();    // ♦ connection opened

// ---- COMMAND + ExecuteScalar ----
NpgsqlCommand cmdScalar =
    new NpgsqlCommand("SELECT COUNT(*) FROM t_student", con);

int total = Convert.ToInt32(cmdScalar.ExecuteScalar());

Console.WriteLine("Total Students = " + total);
```

✓ 2. DataReader (Connected Mode)

```
// ---- COMMAND + DataReader ----
NpgsqlCommand cmdReader =
    new NpgsqlCommand("SELECT id, name FROM t_student", con);

NpgsqlDataReader dr = cmdReader.ExecuteReader();

while (dr.Read())
{
    Console.WriteLine(dr["id"] + " - " + dr["name"]);
}

dr.Close();    // ♦ reader closed
```

✓ 3. ExecuteNonQuery

```
// ---- INSERT / UPDATE ----  
NpgsqlCommand cmdNon =  
    new NpgsqlCommand(  
        "UPDATE t_student SET name='Amit' WHERE id=1", con);  
  
int rows = cmdNon.ExecuteNonQuery();  
  
Console.WriteLine(rows + " record updated");
```

✓ 4. DataAdapter (Disconnected – Partial)

```
// ---- DATA ADAPTER ----  
NpgsqlDataAdapter da =  
    new NpgsqlDataAdapter("SELECT * FROM t_student", con);  
  
DataSet ds = new DataSet();  
  
da.Fill(ds);           // data copied to DataSet  
  
// after fill we can even close connection  
con.Close();           // ♦ connection closed  
  
// use local data  
foreach (DataRow row in ds.Tables[0].Rows)  
{  
    Console.WriteLine(row["name"]);  
}
```

2. Disconnected Architecture

Here data is copied from database and connection is closed.
User works on local copy.

Classes Used

- DataSet
- DataTable
- DataAdapter
- DataView

PostgreSQL Example

```
using Npgsql;
using System.Data;

string connString =

"Host=localhost;Port=5432;Username=postgres;Password=1234;Database=db_
test";

// ---- CONNECTION ----
NpgsqlConnection con = new NpgsqlConnection(connString);

con.Open();          // ♦ 1. Connection Open

// ---- DATA ADAPTER ----
NpgsqlDataAdapter da =
```

```

    new NpgsqlDataAdapter("SELECT * FROM t_student", con);

// ---- DATASET ----
DataSet ds = new DataSet();

da.Fill(ds);      // ♦ 2. Data copied into DataSet

con.Close();      // ♦ 3. Connection Closed

// =====
// Now completely DISCONNECTED MODE
// =====

// ---- DATATABLE ----
DataTable dt = ds.Tables[0];

// ---- DATAVIEW ----
DataView dv = new DataView(dt);
dv.RowFilter = "id > 2";      // example filter

// display local data
foreach (DataRowView row in dv)
{
    Console.WriteLine(row["id"] + " - " + row["name"]);
}

```

Features

- ✓ Works without continuous connection
- ✓ Can hold multiple tables
- ✓ Supports XML
- ✗ Slower than DataReader

✓ Core Components (PostgreSQL + SQL)

Component	PostgreSQL	SQL Server
Connection	NpgsqlConnection	SqlConnection
Command	NpgsqlCommand	SqlCommand
Reader	NpgsqlDataReader	SqlDataReader
Adapter	NpgsqlDataAdapter	SqlDataAdapter

NpgsqlDataReader is used in **connected mode** and reads data **row by row** while the **connection remains open**. It is faster and memory efficient.

NpgsqlDataAdapter is used in **disconnected mode** and loads data into **DataSet** or **DataTable**, allowing data to be filtered, modified, and used offline.



Important Methods

1. ExecuteScalar() – single value

```
SELECT COUNT(*) FROM t_student;
```

```
int count = (int)cmd.ExecuteScalar();
```

2. ExecuteNonQuery() – INSERT/UPDATE/DELETE

```
UPDATE t_student SET name='Ram' WHERE id=1;
```

```
int rows = cmd.ExecuteNonQuery();
```

3. ExecuteReader() – multiple rows

```
SELECT * FROM t_student;
```

```
NpgsqlDataReader dr = cmd.ExecuteReader();  
while(dr.Read())  
{  
    Console.WriteLine(dr["name"]);  
}
```

100 Best Interview Answer

ADO.NET is a database access technology in .NET used to connect applications with PostgreSQL/SQL Server. It supports:

- Connected model → DataReader, Command
 - Disconnected model → DataSet, DataAdapter
 - Methods → ExecuteScalar, ExecuteNonQuery, ExecuteReader
-